QuickPOS

Setup & Deployment Guide

From ZIP file to live app — step by step

What this guide covers

This document walks you through everything needed to get QuickPOS running — from installing Node.js to setting up Supabase as your database, creating shop accounts, and deploying the app so shopkeepers can use it on their phones.

Part	What you will do
Part 1	Install tools on your computer
Part 2	Set up Supabase (the database)
Part 3	Connect the app to Supabase
Part 4	Run the app
Part 5	Create accounts for shopkeepers
Part 6	Deploy online (optional)

Part 1 — Install Tools on Your Computer

Step 1 Extract the ZIP file

Find the downloaded **quickpos.zip** file (usually in your Downloads folder).

Right-click it → **Extract Here**.

You will get a folder called **quickpos**. Remember where it is.

Step 2 Install Node.js

Open your browser and go to: **https://nodejs.org**

Download the **LTS** version (the left green button).

Run the installer and click Next until it finishes.

To confirm it worked, open a Terminal and type:

```
node --version
```

You should see something like: v20.11.0

Step 3 Open Terminal inside the quickpos folder

On **Linux Mint**: right-click the quickpos folder → **Open in Terminal**.

On **Windows**: open the folder → right-click empty space → **Open in PowerShell**.

On **Mac**: right-click the folder → **New Terminal at Folder**.

Part 2 — Set Up Supabase (the Database)

Step 4 Create a free Supabase account

Go to **https://supabase.com** and click **Start your project**.

Sign up with GitHub or email.

Click **New Project**.

Fill in: **Project name** (e.g. quickpos), **Database password** (save this!), **Region** → choose **Central EU** or closest to Kenya.

Click **Create new project** and wait ~2 minutes for it to be ready.

Step 5 Create the database tables

In your Supabase project, click **SQL Editor** in the left sidebar.

Click **New query**.

Paste the entire SQL block below and click **Run** (or press Ctrl+Enter).

```
-- SHOPS table: one row per shop
CREATE TABLE shops (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
owner_id UUID REFERENCES auth.users(id) ON DELETE CASCADE,
name TEXT NOT NULL,
created_at TIMESTAMPTZ DEFAULT now()
);

-- PRODUCTS table: each shop has its own products
CREATE TABLE products (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
shop_id UUID REFERENCES shops(id) ON DELETE CASCADE,
name TEXT NOT NULL,
price NUMERIC(10,2) NOT NULL DEFAULT 0,
barcode TEXT,
stock INTEGER NOT NULL DEFAULT 0,
category TEXT NOT NULL DEFAULT 'General',
emoji TEXT DEFAULT '■',
created_at TIMESTAMPTZ DEFAULT now()
);

-- SALES table: every transaction ever made
```

```
CREATE TABLE sales (
id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
shop_id UUID REFERENCES shops(id) ON DELETE CASCADE,
ts TIMESTAMPTZ NOT NULL DEFAULT now(),
items JSONB NOT NULL,
subtotal NUMERIC(10,2) NOT NULL,
tax NUMERIC(10,2) NOT NULL DEFAULT 0,
total NUMERIC(10,2) NOT NULL,
method TEXT NOT NULL DEFAULT 'cash',
tendered NUMERIC(10,2),
change NUMERIC(10,2)
);
```

Step 6 Enable Row Level Security (RLS)

Still in the SQL Editor, paste this second block and click Run.

This ensures each shopkeeper ONLY sees their own shop data.

```
-- Enable RLS on all tables
ALTER TABLE shops ENABLE ROW LEVEL SECURITY;
ALTER TABLE products ENABLE ROW LEVEL SECURITY;
ALTER TABLE sales ENABLE ROW LEVEL SECURITY;

-- Shops: owner can read their own shop
CREATE POLICY "owner reads own shop"
ON shops FOR SELECT
USING (owner_id = auth.uid());

-- Products: shop owner can read/write their products
CREATE POLICY "owner manages products"
ON products FOR ALL
USING (shop_id IN (SELECT id FROM shops WHERE owner_id = auth.uid()));

-- Sales: shop owner can read/write their sales
CREATE POLICY "owner manages sales"
ON sales FOR ALL
USING (shop_id IN (SELECT id FROM shops WHERE owner_id = auth.uid()));
```

Step 7 Get your Supabase credentials

In your Supabase project, click the **Settings** icon (gear) in the left sidebar.

Click **API**.

You will see two values — copy both and keep them safe:

What to copy	Where to find it
Project URL	Looks like: https://abcxyz123.supabase.co
anon public key	Long string starting with eyJ...

Part 3 — Connect the App to Supabase

Step 8 Paste your credentials into the app

Open the **quickpos** folder on your computer.

Navigate to: **src/lib/supabase.js**

Open it with any text editor (Notepad, VS Code, Gedit).

Find these two lines and replace the placeholder text with your actual values:

```
const SUPABASE_URL = "https://YOUR_PROJECT_ID.supabase.co";

const SUPABASE_ANON = "YOUR_ANON_PUBLIC_KEY";
```

Replace with your actual URL and key, keeping the quotes. Save the file.

■ The anon key is safe to put in the app — it is a public key designed for this use. Never paste your SERVICE ROLE key anywhere in the app.

Part 4 — Run the App

Step 9 Install dependencies (first time only)

In the Terminal you opened inside the quickpos folder, run:

```
npm install
```

This downloads everything the app needs. It takes about 1–2 minutes the first time. You only need to do this once.

Step 10 Start the app

In the same Terminal, run:

```
npm run dev
```

You will see output like this:

```
VITE ready in 400ms

➜ Local: http://localhost:5173/
➜ Network: http://192.168.1.5:5173/
```

Open **http://localhost:5173** in Chrome on your computer.

You should see the QuickPOS login screen.

■ The Network address (e.g. http://192.168.1.5:5173) is what you open on a phone or tablet connected to the same WiFi.

Part 5 — Create Accounts for Shopkeepers

For each shopkeeper you rent the app to, you need to do 3 things:

Step 11 Create the user in Supabase

In your Supabase dashboard, click **Authentication** in the left sidebar.

Click **Users** → **Invite user** (or **Add user**).

Enter the shopkeeper's email and a temporary password.

Click **Create user**.

Copy the **User UID** shown in the users list — you will need it in the next step.

Step 12 Create their shop record

Go to **Table Editor** → click the **shops** table.

Click **Insert row**.

Fill in:

Column	Value to enter
owner_id	The User UID you copied from Step 11
name	The shop's name, e.g. 'Mama Wanjiru Shop'

Step 13 Give them their login details

Tell the shopkeeper their email and password.

They open the app URL in Chrome on their phone.

They log in — they will only see their shop's data, nothing else.

They can change their password from their browser settings if needed.

■ Repeat Steps 11–13 for every shopkeeper. Each one gets their own isolated account and data.

Part 6 — Deploy Online (Optional but Recommended)

Right now the app only runs when your computer is on and npm run dev is running. To make it always available — even when your computer is off — deploy it online for free.

Step 14 Build the production files

Stop the dev server (press Ctrl+C in the Terminal).

Run:

```
npm run build
```

This creates a **dist/** folder with the final app files.

Step 15 Deploy to Netlify (free)

Go to **https://netlify.com** and create a free account.

Click **Add new site** → **Deploy manually**.

Drag and drop the **dist/** folder onto the Netlify page.

Netlify gives you a URL like: **https://random-name-123.netlify.app**

Share this URL with your shopkeepers. That is the app address.

■ After each update to the app, run npm run build again and re-upload the dist/ folder to Netlify.

Quick Reference — Commands

Command	What it does
`npm install`	Install dependencies (first time only)
`npm run dev`	Start the app for testing
`npm run build`	Build the final app for deployment
`npm run preview`	Preview the built app locally